



TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA HÀ NỘI

BÁO CÁO GIỮA KỲ

MỘT SỐ VẤN ĐỀ CHỌN LỌC VỀ TRÍ TUỆ NHÂN TẠO
(MAT3377)

HỆ THỐNG GIÁM SÁT GIAO THÔNG THÔNG MINH VÀ NHẬN DIỆN BIỂN SỐ XE VIỆT NAM

CƠ SỞ LÝ THUYẾT: PHÁT HIỆN ĐỐI TƯỢNG YOLOv8, THEO DÕI ĐA ĐỐI
TƯỢNG BYTETRACK,
NHẬN DẠNG BIỂN SỐ VỚI PADDLEOCR

Nhóm thực hiện:

Nguyễn Hải Đăng -

[22001560]

Mai Hoàng Anh -

[22001538]

Giảng viên hướng dẫn:

TS. Nguyễn Thị Bích Thủy

Hà Nội, tháng 1 năm 2026

Lời cảm ơn

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành nhất đến **Tiến sĩ Nguyễn Thị Bích Thủy** – giảng viên hướng dẫn môn học “Một số vấn đề chọn lọc về Trí tuệ nhân tạo”. Cô đã tận tình hướng dẫn, định hướng nghiên cứu và cung cấp những kiến thức chuyên môn quý báu trong suốt quá trình thực hiện đề tài.

Chúng em cũng xin cảm ơn **Khoa Toán – Cơ – Tin học, Trường Đại học Khoa học Tự nhiên**, Đại học Quốc gia Hà Nội đã tạo điều kiện về cơ sở vật chất, tài liệu học tập và môi trường nghiên cứu thuận lợi để nhóm hoàn thành đề tài này.

Đồng thời, nhóm xin bày tỏ lòng biết ơn đến các bạn sinh viên cùng lớp đã đóng góp ý kiến, chia sẻ kinh nghiệm và hỗ trợ trong quá trình thu thập dữ liệu, triển khai hệ thống.

Cuối cùng, nhóm xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, khích lệ trong suốt quá trình học tập và nghiên cứu.

Mặc dù đã cố gắng hết sức, báo cáo không tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét từ quý thầy cô và các bạn để hoàn thiện hơn nữa kiến thức và kỹ năng nghiên cứu.

Hà Nội, tháng 1 năm 2026
Nhóm thực hiện

Tóm tắt

Báo cáo giữa kỳ này trình bày cơ sở lý thuyết cho đề tài “**Hệ thống giám sát giao thông thông minh và nhận diện biển số xe Việt Nam**”. Nội dung báo cáo tập trung vào ba phần chính:

Chương 1: Mở đầu – Trình bày bối cảnh giao thông Việt Nam năm 2025, những thách thức trong việc áp dụng các hệ thống giám sát thông minh, bài toán nhận diện biển số xe, mục tiêu và phạm vi nghiên cứu của đề tài.

Chương 2: Cơ sở lý thuyết về phát hiện và theo dõi đối tượng – Trình bày nền tảng toán học của bài toán phát hiện đối tượng (Object Detection), kiến trúc và cơ chế hoạt động của mô hình YOLOv8 (bao gồm Anchor-free, C2f Module, SPPF, Task Aligned Assigner), các hàm mất mát (CIoU Loss, VFL), và thuật toán theo dõi đa đối tượng ByteTrack với Bộ lọc Kalman.

Chương 3: Cơ sở lý thuyết về nhận diện biển số xe – Trình bày tổng quan về hệ thống ALPR (Automatic License Plate Recognition), cấu trúc biển số xe Việt Nam, kiến trúc PaddleOCR với CRNN và CTC Loss, các kỹ thuật tiền xử lý ảnh (CLAHE, Sharpening), và cơ chế Validation & Retry để đảm bảo chuỗi OCR đúng format.

Báo cáo này đặt nền móng lý thuyết vững chắc cho việc triển khai ở giai đoạn cuối kỳ và các bước hoàn thiện hệ thống.

Từ khóa: YOLOv8, ByteTrack, ALPR, PaddleOCR, Kalman Filter, Object Detection, Multi-Object Tracking, License Plate Recognition.

Mục lục

Lời cảm ơn	ii
Tóm tắt	iii
Danh sách các từ viết tắt	viii
1 Mở đầu	1
1.1 Đặt vấn đề	1
1.1.1 Bối cảnh giao thông Việt Nam năm 2025	1
1.1.2 Bài toán nhận diện biển số xe	1
1.1.3 Định hướng nghiên cứu	2
1.2 Mục tiêu của đề tài	2
1.2.1 Module phát hiện và theo dõi phương tiện	2
1.2.2 Module phát hiện vi phạm	2
1.2.3 Module nhận diện biển số xe	2
1.3 Phạm vi nghiên cứu	3
1.3.1 Dữ liệu	3
1.3.2 Công nghệ sử dụng	3
1.3.3 Giới hạn	3
1.4 Cấu trúc báo cáo	3
2 Cơ sở lý thuyết: Phát hiện và theo dõi đối tượng	4
2.1 Tổng quan về bài toán Phát hiện đối tượng	4
2.1.1 Giới thiệu chung	4
2.1.2 Định nghĩa toán học	4
2.1.3 Thách thức của bài toán	5
2.2 Cơ sở lý thuyết về mô hình YOLO	5
2.2.1 Hợp nhất bài toán thành một mạng duy nhất	5
2.2.2 Cơ chế chia lưới và Biểu diễn đầu ra	5
2.2.3 Kiến trúc mạng tổng quát	6
2.2.4 Sự tiến hóa và Lựa chọn YOLOv8	6
2.3 Mô hình YOLOv8 chi tiết	7
2.3.1 Cơ chế Anchor-free	7
2.3.2 Các thành phần kiến trúc đặc trưng	7
2.3.3 Hàm mất mát (Loss Function)	8
2.4 Theo dõi đa đối tượng (Multi-Object Tracking)	9
2.4.1 Bộ lọc Kalman (Kalman Filter)	9

2.4.2	Liên kết dữ liệu (Data Association)	11
2.4.3	Thuật toán ByteTrack	12
2.5	Xử lý ảnh và Hình học tính toán	14
2.5.1	Không gian màu HSV trong nhận diện tín hiệu đèn	14
2.5.2	Hình học tính toán trong phát hiện vi phạm	14
3	Cơ sở lý thuyết: Nhận diện biển số xe	16
3.1	Tổng quan về ALPR	16
3.2	Biển số xe Việt Nam	16
3.2.1	Các định dạng biển số	16
3.2.2	Thách thức nhận diện biển số Việt Nam	17
3.3	PaddleOCR - Nhận dạng ký tự	17
3.3.1	Giới thiệu PaddleOCR	17
3.3.2	Pipeline nhận dạng văn bản trong PaddleOCR	17
3.3.3	Rectification - Nắn thẳng biển số	17
3.3.4	Kiến trúc SVTR - Scene Text Recognition	19
3.3.5	Kiến trúc CRNN (Legacy)	20
3.3.6	CTC Decoder - Giải mã chuỗi ký tự	21
3.4	Tiền xử lý ảnh biển số	22
3.4.1	CLAHE - Cân bằng histogram thích ứng	22
3.4.2	Sharpening - Làm sắc nét	23
3.4.3	Quy trình tiền xử lý hoàn chỉnh	24
3.5	So sánh các phiên bản YOLOv8	24
3.6	Cơ chế Validation & Retry	24
3.6.1	Vấn đề với OCR đơn frame	24
3.6.2	Giải pháp: Format Validation	24
3.6.3	Retry Mechanism	24
3.7	Character Correction cho biển số Việt Nam	25
3.7.1	Các lỗi phổ biến của OCR	25
3.7.2	Logic hiệu chỉnh dựa trên định dạng	25
3.8	Hai phương pháp triển khai ALPR	26
3.8.1	Phương pháp 1: Two-Stage ALPR (Hai giai đoạn)	26
3.8.2	Phương pháp 2: One-Stage Semi-supervised (Một giai đoạn)	26
3.8.3	So sánh hai phương pháp	27

Danh sách hình vẽ

2.1	Cơ chế chia lưới và biểu diễn đầu ra của YOLO	6
2.2	Chu trình hoạt động của Bộ lọc Kalman	10
2.3	Minh họa quy trình liên kết hai giai đoạn của ByteTrack	13
2.4	So sánh góc chuyển động $\vec{v}$ với góc tham chiếu $\vec{r}$ (ngưỡng 50°). Vùng màu xanh lá: đi thẳng, vùng ngoài: rẽ trái/phải.	15
3.1	Pipeline tổng quan của hệ thống ALPR	16
3.2	Pipeline nhận dạng văn bản trong PaddleOCR	17
3.3	Minh họa quá trình Rectification biển số xe	18
3.4	Kiến trúc Spatial Transformer Network	18
3.5	TPS transformation nắn thẳng văn bản cong	19
3.6	Kiến trúc tổng quan của SVTR (nguồn: Du et al., 2022)	19
3.7	Kiến trúc SVTR trong PaddleOCR	20
3.8	So sánh Local Mixing và Global Mixing trong SVTR	20
3.9	Minh họa cắt ảnh thành các lát dọc trong SVTR	21
3.10	Kiến trúc CRNN cho OCR trong PaddleOCR	21

Danh sách bảng

3.1	Các định dạng biển số xe Việt Nam	16
3.2	So sánh kiến trúc SVTR và CRNN	21
3.3	So sánh các phiên bản YOLOv8	24
3.4	Bảng hiệu chỉnh ký tự phổ biến	25

Danh sách các từ viết tắt

AI	Artificial Intelligence (Trí tuệ nhân tạo)
ALPR	Automatic License Plate Recognition (Nhận dạng biển số tự động)
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
CPU	Central Processing Unit (Bộ xử lý trung tâm)
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
CIoU	Complete Intersection over Union (Hàm mất mát định vị)
DFL	Distribution Focal Loss
EMA	Exponential Moving Average
FN	False Negative (Âm tính giả - Bỏ sót đối tượng)
FP	False Positive (Dương tính giả - Bắt nhầm đối tượng)
GPU	Graphics Processing Unit (Bộ xử lý đồ họa)
HSV	Hue, Saturation, Value (Không gian màu)
IoU	Intersection over Union (Chỉ số Giao trên Hợp)
LSTM	Long Short-Term Memory
MOT	Multi-Object Tracking (Theo dõi đa đối tượng)
OCR	Optical Character Recognition (Nhận dạng ký tự quang học)
R-CNN	Region-based Convolutional Neural Network
RGB	Red, Green, Blue (Không gian màu đỏ-lục-lam)
ROI	Region of Interest (Vùng quan tâm)
TP	True Positive (Dương tính thật - Dự đoán đúng)
VFL	Varifocal Loss

VNLP Vietnamese License Plate (Biển số xe Việt Nam)

YOLO You Only Look Once (Mô hình phát hiện đối tượng)

Chương 1

Mở đầu

1.1 Đặt vấn đề

1.1.1 Bối cảnh giao thông Việt Nam năm 2025

Bước sang năm 2025, quá trình đô thị hóa và chuyển đổi số tại Việt Nam đang diễn ra mạnh mẽ hơn bao giờ hết. Tại các thành phố lớn như Hà Nội và TP. Hồ Chí Minh, chính quyền đã bắt đầu triển khai thí điểm các hệ thống giao thông thông minh (Intelligent Transportation System - ITS) và camera phạt nguội tích hợp trí tuệ nhân tạo tại các nút giao trọng điểm. Những nỗ lực này bước đầu đã mang lại hiệu quả trong việc giảm ã và nâng cao ý thức người tham gia giao thông.

Tuy nhiên, việc áp dụng đại trà các hệ thống này vẫn gặp nhiều thách thức lớn:

- **Chi phí đầu tư cao:** Các hệ thống AI nhập ngoại hoặc các giải pháp phần cứng chuyên dụng đòi hỏi ngân sách lớn để triển khai và vận hành.
- **Đặc thù giao thông Việt Nam:** Mật độ xe máy dày đặc, hiện tượng giao thông hỗn hợp (mixed traffic) và hành vi di chuyển “điền vào chỗ trống” khiến các mô hình AI chuẩn quốc tế thường xuyên gặp sai sót hoặc bỏ lọt vi phạm.
- **Nhu cầu làm chủ công nghệ:** Việc tùy biến và tích hợp sâu vào hạ tầng sẵn có đòi hỏi phải làm chủ được công nghệ lõi.

1.1.2 Bài toán nhận diện biển số xe

Song song với giám sát vi phạm, nhận diện biển số xe tự động (Automatic License Plate Recognition - ALPR) là một trong những ứng dụng quan trọng của thị giác máy tính trong lĩnh vực giao thông thông minh. Tại Việt Nam, nhu cầu về hệ thống ALPR ngày càng tăng cao trong các ứng dụng thực tế như:

- Hệ thống thu phí tự động (ETC - Electronic Toll Collection)
- Quản lý bãi đỗ xe thông minh
- Giám sát giao thông và ghi nhận vi phạm
- Kiểm soát an ninh tại các cổng ra vào

Biển số xe Việt Nam có định dạng đặc thù với các ký tự chữ và số, được phân chia theo tỉnh/thành phố và loại phương tiện. Điều này đặt ra thách thức riêng cho các hệ thống ALPR quốc tế khi áp dụng vào thực tế Việt Nam do:

- Định dạng biển số khác biệt so với tiêu chuẩn quốc tế

- Chất lượng ảnh từ camera giám sát không đồng đều
- Điều kiện thời tiết và ánh sáng đa dạng
- Font chữ và kích thước biển số đặc thù

1.1.3 Định hướng nghiên cứu

Xuất phát từ thực tế trên, nhóm nghiên cứu lựa chọn đề tài “**Hệ thống giám sát giao thông thông minh và nhận diện biển số xe Việt Nam**” với mục tiêu xây dựng một hệ thống hoàn chỉnh có khả năng:

1. Chạy trên phần cứng phổ thông với khả năng xử lý thời gian thực
2. Đảm bảo độ tin cậy trong việc nhận diện các loại phương tiện đặc thù
3. Tự động phát hiện các hành vi vi phạm giao thông phức tạp
4. Nhận diện và đọc được biển số xe Việt Nam

Hệ thống sử dụng mô hình YOLOv8 tiên tiến, được tinh chỉnh (fine-tune) trên dữ liệu giao thông nội địa, kết hợp với các thuật toán hậu xử lý thông minh và công nghệ OCR hiện đại để giải quyết bài toán giám sát giao thông với chi phí thấp và phù hợp triển khai.

1.2 Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống giám sát giao thông hoàn chỉnh với các khả năng sau:

1.2.1 Module phát hiện và theo dõi phương tiện

- **Nhận diện đa lớp phương tiện:** Phân loại 5 loại phương tiện phổ biến trong dòng giao thông hỗn hợp: Xe máy, Xe đạp, Ô tô con, Xe buýt và Xe tải.
- **Theo dõi và đếm lưu lượng:** Sử dụng YOLOv8 với thuật toán tracking ByteTrack tích hợp để theo dõi quỹ đạo di chuyển, đếm lưu lượng xe theo từng loại và từng làn đường.

1.2.2 Module phát hiện vi phạm

- **Giám sát tín hiệu giao thông:** Tự động định vị và nhận diện trạng thái màu sắc của đèn giao thông (Xanh/Đỏ/Vàng) bằng kỹ thuật xử lý ảnh trên không gian màu HSV.
- **Phát hiện vi phạm tự động:** Kết hợp logic máy tính để phát hiện các hành vi: Vượt đèn đỏ (dựa trên phân tích hướng di chuyển) và Di sai làn đường quy định.

1.2.3 Module nhận diện biển số xe

- **Phát hiện biển số (Plate Detection):** Huấn luyện mô hình YOLOv8 để phát hiện vị trí biển số trong ảnh/video.
- **Nhận dạng ký tự (OCR):** Sử dụng PaddleOCR kết hợp các kỹ thuật tiền xử lý (CLAHE, Sharpening) để nhận dạng các ký tự trên biển số.
- **Cơ chế ổn định đầu ra:** Xây dựng Validation & Retry mechanism để đảm

bảo chuỗi OCR đúng format biển số Việt Nam.

1.3 Phạm vi nghiên cứu

1.3.1 Dữ liệu

- **Dữ liệu phương tiện:** Sử dụng bộ dữ liệu thực tế tự quay chụp và thu thập để đảm bảo mô hình học được các đặc trưng của phương tiện tại Việt Nam (hơn 16.000 ảnh với 65.000 bounding box).
- **Dữ liệu biển số:** Sử dụng bộ dữ liệu VNLP (Vietnamese License Plate) để huấn luyện mô hình phát hiện biển số.

1.3.2 Công nghệ sử dụng

- **Phát hiện đối tượng:** YOLOv8n (phiên bản nano) – gọn nhẹ
- **Theo dõi đa đối tượng:** ByteTrack (tích hợp trong YOLOv8)
- **Xử lý tín hiệu đèn:** Thuật toán trên không gian màu HSV
- **Phát hiện vi phạm:** Thuật toán hình học với Direction Fusion
- **OCR biển số:** PaddleOCR với kiến trúc CRNN + CTC
- **Tiền xử lý ảnh:** CLAHE và Sharpening

1.3.3 Giới hạn

- **Loại camera:** Hệ thống tập trung xử lý video từ camera giám sát cố định (CCTV) với góc quay từ trên cao.
- **Điều kiện ánh sáng:** Hoạt động tốt trong điều kiện ánh sáng ban ngày và ban đêm có đèn đường.
- **Loại biển số:** Hỗ trợ biển số xe Việt Nam cho cả ô tô (7-8 ký tự) và xe máy (8-9 ký tự), tập trung vào biển trắng 1 dòng.
- **Đầu vào:** Video giao thông từ camera cố định.
- **Đầu ra:** Thông tin phương tiện, cảnh báo vi phạm và chuỗi ký tự biển số đã được nhận dạng.

1.4 Cấu trúc báo cáo

Báo cáo giữa kỳ được trình bày với bố cục như sau:

- **Chương 1: Mở đầu.** Trình bày bối cảnh giao thông Việt Nam, lý do chọn đề tài, mục tiêu và phạm vi nghiên cứu.
- **Chương 2: Cơ sở lý thuyết – Phát hiện và theo dõi đối tượng.** Trình bày nền tảng toán học của mô hình YOLOv8, cơ chế tracking tích hợp (ByteTrack) và các kỹ thuật xử lý ảnh trong nhận diện tín hiệu đèn và hình học tính toán.
- **Chương 3: Cơ sở lý thuyết – Nhận diện biển số xe.** Trình bày kiến trúc PaddleOCR, các kỹ thuật tiền xử lý ảnh (CLAHE, Sharpening) và cấu trúc biển số xe Việt Nam.

Các phần tiếp theo (Phương pháp thực hiện, Thực nghiệm và Đánh giá, Kết luận) sẽ được trình bày trong báo cáo cuối kỳ.

Chương 2

Cơ sở lý thuyết: Phát hiện và theo dõi đối tượng

2.1 Tổng quan về bài toán Phát hiện đối tượng

2.1.1 Giới thiệu chung

Phát hiện đối tượng (Object Detection) là một bài toán nền tảng và thiết yếu trong lĩnh vực Thị giác máy tính (Computer Vision). Mục đích chính của bài toán là xác định sự hiện diện của các thực thể quan tâm trong một hình ảnh kỹ thuật số hoặc video, đồng thời định vị chính xác vị trí và gán nhãn phân loại cho từng thực thể đó.

Khác với bài toán phân loại ảnh (Image Classification) đơn thuần chỉ gán một nhãn duy nhất cho toàn bộ bức ảnh, phát hiện đối tượng giải quyết đồng thời hai nhiệm vụ phức tạp:

1. **Định vị (Localization):** Xác định vị trí của đối tượng thông qua một khung bao (Bounding Box).
2. **Phân loại (Classification):** Xác định danh mục (class) của đối tượng nằm trong khung bao đó.

Bài toán này đóng vai trò tiền đề cho nhiều ứng dụng thực tế quan trọng như: xe tự hành (autonomous driving), hệ thống giám sát an ninh, chẩn đoán y tế qua hình ảnh và tương tác người-máy.

2.1.2 Định nghĩa toán học

Về mặt hình thức, xét một ảnh đầu vào $I \in \mathbb{R}^{H \times W \times C}$, trong đó H, W, C lần lượt là chiều cao, chiều rộng và số kênh màu của ảnh. Mục tiêu của mô hình $f(I)$ là dự đoán một tập hợp các bounding box $B = \{b_1, b_2, \dots, b_N\}$, trong đó mỗi b_i đại diện cho một đối tượng được phát hiện và được định nghĩa bởi vector:

$$b_i = (x_i, y_i, w_i, h_i, c_i, p_i) \quad (2.1)$$

Các tham số được giải thích cụ thể như sau:

- (x_i, y_i) : Tọa độ tâm của bounding box, thường được chuẩn hóa theo kích thước ảnh hoặc so với lưới đặc trưng (feature grid).

- (w_i, h_i) : Chiều rộng và chiều cao của bounding box. Trong các mô hình hiện đại, giá trị này thường được dự đoán dưới dạng độ lệch (offset) log-space so với các khung neo (anchor) định trước.
- $c_i \in \{1, \dots, K\}$: Nhãn lớp của đối tượng, thuộc một trong K danh mục quan tâm (ví dụ: người, xe, chó, mèo).
- $p_i \in [0, 1]$: Độ tin cậy (confidence score), phản ánh xác suất mà mô hình tin rằng bounding box này chứa một đối tượng thực sự thuộc lớp c_i .

2.1.3 Thách thức của bài toán

Việc ánh xạ từ không gian pixel sang không gian ngữ nghĩa của các bounding box gặp phải nhiều thách thức lớn, bao gồm:

- **Sự biến thiên về tỷ lệ (Scale Variation)**: Các đối tượng trong ảnh có thể xuất hiện với kích thước rất khác nhau, từ vài pixel đến chiếm toàn bộ khung hình.
- **Sự che khuất (Occlusion)**: Các đối tượng có thể bị che khuất một phần bởi các đối tượng khác hoặc nền, làm mất đi các đặc trưng quan trọng để nhận dạng.
- **Điều kiện ánh sáng**: Sự thay đổi về ánh sáng, bóng đổ, phản chiếu ảnh hưởng đến chất lượng ảnh đầu vào.
- **Giao thông hỗn hợp**: Đặc thù tại Việt Nam với nhiều loại phương tiện di chuyển xen kẽ nhau.

2.2 Cơ sở lý thuyết về mô hình YOLO

2.2.1 Hợp nhất bài toán thành một mạng duy nhất

Trước khi kiến trúc YOLO (You Only Look Once) ra đời, các hệ thống phát hiện đối tượng kinh điển thường xử lý bài toán theo quy trình hai giai đoạn tách biệt (Two-Stage Detectors). Các phương pháp như R-CNN sử dụng thuật toán đề xuất vùng (Region Proposal) như Selective Search để tìm kiếm các vùng ứng viên, sau đó mới áp dụng bộ phân loại trên từng vùng này. Quy trình này tốn kém về mặt tính toán và khó tối ưu hóa toàn cục.

YOLO, được giới thiệu bởi Redmon et al. (2015), đã thay đổi hoàn toàn tư duy này bằng cách tái định nghĩa việc phát hiện đối tượng thành một bài toán **hồi quy duy nhất** (Single Regression Problem). Thay vì quét ảnh nhiều lần, YOLO chỉ cần một lần truyền thẳng (forward pass) qua mạng nơ-ron tích chập (CNN) để dự đoán đồng thời cả vị trí khung bao (bounding box) và xác suất lớp đối tượng. Điều này cho phép mô hình nhìn thấy ngữ cảnh toàn cục của bức ảnh, giảm thiểu các sai sót do nhầm lẫn hậu cảnh (background).

2.2.2 Cơ chế chia lưới và Biểu diễn đầu ra

Cơ chế hoạt động cốt lõi của mọi phiên bản YOLO dựa trên việc chia ảnh đầu vào thành một lưới kích thước $S \times S$. Quy tắc gán nhãn được định nghĩa như sau:

- Nếu tâm (center) của một đối tượng thực tế rơi vào ô lưới nào, ô lưới đó chịu trách nhiệm phát hiện đối tượng đó.

- Mỗi ô lưới sẽ dự đoán B khung bao dự kiến, cùng với độ tin cậy (confidence) cho mỗi khung bao.

Về mặt toán học, độ tin cậy C_{pred} được định nghĩa là tích của xác suất có đối tượng và độ chồng lấn (IoU) với thực tế:

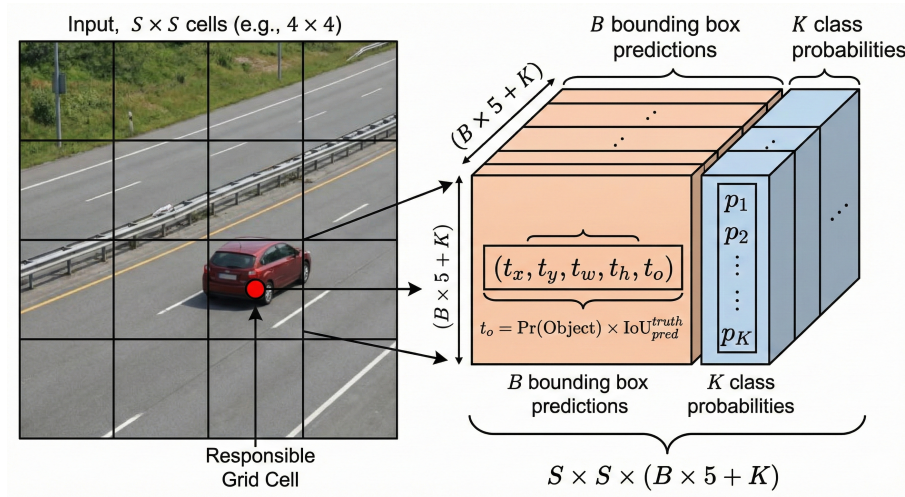
$$C_{pred} = Pr(\text{Object}) \times \text{IoU}_{pred}^{truth} \quad (2.2)$$

Nếu không có đối tượng trong ô lưới, $Pr(\text{Object}) = 0$, dẫn đến $C_{pred} = 0$.

Đầu ra của mạng là một tensor 3 chiều có kích thước $S \times S \times (B \times 5 + K)$, trong đó K là số lớp đối tượng. Mỗi vector dự đoán bao gồm 5 tham số cơ bản:

$$\hat{y} = (t_x, t_y, t_w, t_h, t_o) \quad (2.3)$$

Trong đó (t_x, t_y) là tọa độ tâm cục bộ trong ô lưới, (t_w, t_h) là kích thước khung bao được chuẩn hóa, và t_o là độ tin cậy.



Hình 2.1: Cơ chế chia lưới và biểu diễn đầu ra của YOLO

2.2.3 Kiến trúc mạng tổng quát

Hầu hết các phiên bản YOLO hiện đại đều tuân theo kiến trúc Meta-architecture gồm 3 thành phần chính:

1. **Backbone (Xương sống):** Là một mạng CNN đóng vai trò trích xuất đặc trưng từ ảnh đầu vào ở các mức độ phân giải khác nhau.
2. **Neck (Cổ):** Sử dụng các kỹ thuật như Feature Pyramid Network (FPN) hoặc Path Aggregation Network (PANet) để trộn và kết hợp các đặc trưng từ Backbone, giúp mô hình nhận diện tốt cả đối tượng nhỏ và lớn.
3. **Head (Đầu):** Thực hiện nhiệm vụ cuối cùng là dự đoán lớp và tọa độ khung bao trên các bản đồ đặc trưng đã được tổng hợp.

2.2.4 Sự tiến hóa và Lựa chọn YOLOv8

Trải qua quá trình phát triển, YOLO đã có những bước tiến vượt bậc:

- **YOLOv1-v3:** Tập trung vào triển khai thời gian thực, giới thiệu khái niệm Anchor Box để ổn định việc huấn luyện.

- **YOLOv4-v5:** Tối ưu hóa mạnh mẽ về kỹ thuật huấn luyện (Training Tricks) như Mosaic Augmentation và cải tiến kiến trúc với CSPNet để tăng cường luồng gradient.
- **YOLOv6-v8 (2022-2023):** Đánh dấu sự chuyển dịch sang kiến trúc **Anchor-free** (không dùng khung neo) và **Decoupled Head** (tách riêng nhánh phân loại và hồi quy).

Lý do lựa chọn YOLOv8: Đề tài này lựa chọn **YOLOv8** (Ultralytics, 2023) làm mô hình chủ đạo. Đây là phiên bản không quá nặng, kế thừa tinh hoa của các thế hệ trước nhưng sở hữu các cải tiến vượt trội phù hợp cho bài toán giao thông tại Việt Nam:

- **Anchor-free:** Loại bỏ nhu cầu thiết kế kích thước anchor box thủ công, giúp mô hình linh hoạt hơn với các đối tượng có tỷ lệ kích thước biến thiên lớn (như xe máy chở hàng công kênh, xe buýt).
- **C2f Module:** Thay thế module C3 cũ, giúp mô hình nhẹ hơn nhưng trích xuất đặc trưng phong phú hơn nhờ các luồng gradient song song.
- **Task Aligned Assigner:** Cơ chế gán nhãn động thông minh, giúp cân bằng tốt hơn giữa mẫu dương và mẫu âm trong quá trình huấn luyện.

2.3 Mô hình YOLOv8 chi tiết

YOLOv8 là mô hình phát hiện đối tượng một giai đoạn (one-stage detector) tiên tiến, coi bài toán phát hiện đối tượng là một bài toán hồi quy trực tiếp từ các pixel của ảnh sang tọa độ bounding box và xác suất lớp.

2.3.1 Cơ chế Anchor-free

Khác với các thế hệ trước dựa trên Anchor Box (khung neo), YOLOv8 áp dụng phương pháp **Anchor-free**. Thay vì dự đoán độ lệch (offset) so với anchor box, mô hình dự đoán trực tiếp khoảng cách từ tâm của ô lưới (grid cell) đến 4 cạnh của bounding box thực tế.

Giả sử tâm của một đối tượng rơi vào ô lưới tại tọa độ (c_x, c_y) , mô hình sẽ dự đoán vector 4 chiều $t = (l, t, r, b)$ đại diện cho khoảng cách tới cạnh trái, trên, phải, và dưới.

2.3.2 Các thành phần kiến trúc đặc trưng

Module C2f (Cross Stage Partial with 2 flow)

YOLOv8 giới thiệu module C2f để thay thế cho module C3 trong các phiên bản trước. C2f được thiết kế dựa trên ý tưởng của kiến trúc ELAN (Efficient Layer Aggregation Network), giúp cải thiện luồng gradient thông tin trong mạng nơ-ron mà không làm tăng đáng kể chi phí tính toán.

Cơ chế hoạt động của C2f bao gồm hai bước:

1. Tách tensor đầu vào thành hai phần theo kênh (channel-wise split).
2. Một phần đi qua chuỗi các lớp Bottleneck, phần còn lại đi thẳng (shortcut). Cuối cùng, tất cả các nhánh được nối (concatenate) lại với nhau.

Công thức tổng quát cho đầu ra Y của khối C2f có thể mô tả như sau:

$$Y = \text{Conv}_{1 \times 1}([X_1, \text{Block}_1(X_2), \text{Block}_2(\text{Block}_1(X_2)), \dots]) \quad (2.4)$$

Kiến trúc này giúp mô hình học được các đặc trưng phong phú hơn (richer features) nhờ việc kết hợp thông tin ở nhiều độ sâu khác nhau.

Module SPPF (Spatial Pyramid Pooling Fast)

Nằm ở cuối phần Backbone, module SPPF có nhiệm vụ trích xuất các đặc trưng ngữ cảnh không gian (spatial context features) ở nhiều tỷ lệ khác nhau. Thay vì sử dụng các bộ lọc có kích thước lớn gây tốn kém tính toán, SPPF thực hiện tuần tự 3 lần phép gộp cực đại (Max Pooling) với kích thước 5×5 .

Đầu ra là sự kết hợp (concatenation) của đầu vào gốc và đầu ra của 3 lần pooling:

$$\text{Out} = \text{Concat}(X, \text{MaxPool}(X), \text{MaxPool}^2(X), \text{MaxPool}^3(X)) \quad (2.5)$$

Điều này giúp mô hình nhận diện tốt các đối tượng có kích thước thay đổi lớn (như xe buýt so với xe máy) và thuận lợi cho triển khai.

Chiến lược gán nhãn Task Aligned Assigner

Một trong những cải tiến quan trọng nhất của YOLOv8 là loại bỏ cơ chế gán nhãn dựa trên IoU tĩnh (static IoU-based assignment) và thay thế bằng **Task Aligned Assigner**. Chiến lược này chọn lọc các mẫu dương (positive samples) dựa trên sự cân bằng giữa điểm phân loại và chất lượng định vị.

Điểm số căn chỉnh (alignment score) t cho mỗi anchor point được tính như sau:

$$t = s^\alpha \times u^\beta \quad (2.6)$$

Trong đó:

- s : Điểm số phân loại dự đoán (predicted classification score).
- u : Chỉ số IoU giữa box dự đoán và box thực tế.
- α và β : Các tham số điều trọng số (thường $\alpha = 0.5, \beta = 6.0$).

Chỉ những mẫu có điểm t cao nhất mới được chọn làm mẫu dương để huấn luyện. Cơ chế này giúp mô hình tập trung vào các dự đoán chất lượng cao, nơi mà "phân loại đúng" đi đôi với "định vị chính xác".

2.3.3 Hàm mất mát (Loss Function)

Quá trình huấn luyện YOLOv8 nhằm tối thiểu hóa hàm mất mát tổng hợp L_{total} , bao gồm hai thành phần chính: mất mát về định vị (L_{box}) và mất mát về phân loại (L_{cls}).

$$L_{total} = \lambda_{box} L_{box} + \lambda_{cls} L_{cls} + \lambda_{dfl} L_{dfl} \quad (2.7)$$

Mất mát định vị (Box Loss)

YOLOv8 sử dụng hàm mất mát **CIoU (Complete IoU)** để đo lường sai số giữa khung dự đoán (B_{pred}) và khung thực tế (B_{gt}). CIoU khắc phục nhược điểm của IoU truyền thống bằng cách xem xét thêm khoảng cách tâm và tỷ lệ khung hình.

Công thức CIoU được định nghĩa như sau:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (2.8)$$

Trong đó:

- $IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|}$ là chỉ số giao trên hợp.
- $\rho(\cdot)$ là khoảng cách Euclidean giữa tâm của hai bounding box $\mathbf{b}$ và $\mathbf{b}^{gt}$.
- c là đường chéo của bao lồi nhỏ nhất chứa cả hai box.
- α là tham số cân bằng trọng số.
- v đo lường sự tương đồng về tỷ lệ khung hình (aspect ratio consistency):

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2 \quad (2.9)$$

Ngoài ra, YOLOv8 còn kết hợp **Distribution Focal Loss (DFL)** để tối ưu hóa phân phối xác suất của các cạnh bounding box, giúp mô hình hội tụ ổn định hơn khi dự đoán các box có biên mờ.

Mất mát phân loại (Classification Loss)

Đối với nhánh phân loại, YOLOv8 sử dụng **Varifocal Loss (VFL)** hoặc Binary Cross Entropy (BCE). VFL giúp cân bằng giữa các mẫu dương (positive samples) và mẫu âm (negative samples) khó học:

$$VFL(p, q) = \begin{cases} -q(q \log(p) + (1 - q) \log(1 - p)) & \text{nếu } q > 0 \\ -\alpha p^\gamma \log(1 - p) & \text{nếu } q = 0 \end{cases} \quad (2.10)$$

Trong đó p là xác suất dự đoán và q là điểm IoU mục tiêu (target IoU score).

2.4 Theo dõi đa đối tượng (Multi-Object Tracking)

Bài toán Theo dõi đa đối tượng (Multi-Object Tracking - MOT) nhằm mục đích ước lượng quỹ đạo của các đối tượng trong video. Giả sử tại mỗi khung hình t , ta nhận được một tập hợp các detection $D_t = \{d_1, d_2, \dots, d_n\}$. Mục tiêu là gán một ID duy nhất k cho mỗi chuỗi detection $(d_{t_1}^k, d_{t_2}^k, \dots)$ tương ứng với cùng một thực thể thực tế.

Hệ thống sử dụng phương pháp **Tracking-by-Detection** kết hợp với thuật toán **ByteTrack**, trong đó thành phần cốt lõi là Bộ lọc Kalman để ước lượng trạng thái và thuật toán Hungarian để liên kết dữ liệu.

2.4.1 Bộ lọc Kalman (Kalman Filter)

Bộ lọc Kalman là một thuật toán đệ quy tối ưu để ước lượng trạng thái của một hệ thống động lực học từ các phép đo nhiễu. Trong bài toán tracking phương tiện, chúng tôi mô hình hóa chuyển động của xe là chuyển động đều (constant velocity model).

Không gian trạng thái (State Space)

Trạng thái của một phương tiện tại thời điểm k được biểu diễn bởi vector 8 chiều $\mathbf{x}_k$:

$$\mathbf{x}_k = [u_k, v_k, a_k, h_k, \dot{u}_k, \dot{v}_k, \dot{a}_k, \dot{h}_k]^T \quad (2.11)$$

Trong đó:

- (u_k, v_k) : Tọa độ tâm của bounding box.
- a_k : Tỷ lệ khung hình (aspect ratio w/h).
- h_k : Chiều cao của bounding box.
- $\dot{u}_k, \dot{v}_k, \dot{a}_k, \dot{h}_k$: Vận tốc biến thiên tương ứng của các tham số trên.

Quá trình Dự đoán (Prediction)

Dựa trên trạng thái ở thời điểm trước đó $k - 1$, bộ lọc dự đoán trạng thái tiên nghiệm (a priori) tại thời điểm k :

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} \quad (2.12)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad (2.13)$$

Trong đó:

- $\mathbf{F}_k$: Ma trận chuyển trạng thái (state transition matrix), mô tả mô hình vật lý của chuyển động.
- $\mathbf{P}_{k|k-1}$: Ma trận hiệp phương sai số dự đoán.
- $\mathbf{Q}_k$: Ma trận hiệp phương sai nhiễu của quá trình (process noise).

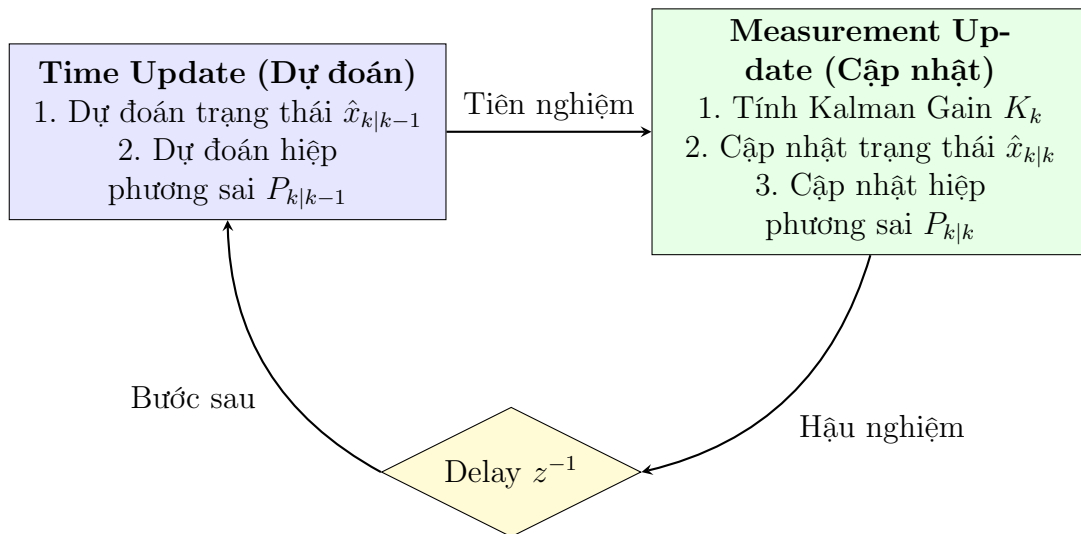
Quá trình Cập nhật (Update)

Khi nhận được phép đo mới (measurement) $\mathbf{z}_k$ từ YOLOv8 (tọa độ bounding box $[u, v, a, h]^T$), bộ lọc sẽ hiệu chỉnh trạng thái dự đoán để có trạng thái hậu nghiệm (a posteriori) chính xác hơn:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (2.14)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1}) \quad (2.15)$$

Trong đó $\mathbf{K}_k$ là độ lợi Kalman (Kalman Gain), $\mathbf{H}_k$ là ma trận quan sát, và $\mathbf{R}_k$ là nhiễu đo lường.



Hình 2.2: Chu trình hoạt động của Bộ lọc Kalman

2.4.2 Liên kết dữ liệu (Data Association)

Liên kết dữ liệu là bước quyết định trong hệ thống theo dõi, đóng vai trò cầu nối giữa các đầu ra phát hiện rời rạc (discrete detections) và các quỹ đạo liên tục (continuous trajectories). Giả sử tại thời điểm t , ta có tập hợp M quỹ đạo hiện hành $T = \{t_1, t_2, \dots, t_M\}$ và tập hợp N đối tượng vừa được phát hiện $D = \{d_1, d_2, \dots, d_N\}$.

Bài toán đặt ra là tìm một ánh xạ ghép cặp 1-1 tối ưu giữa T và D sao cho tổng độ sai lệch (chi phí) là nhỏ nhất. Vấn đề này được mô hình hóa dưới dạng bài toán **Phân công Tuyến tính (Linear Assignment Problem)** trên đồ thị hai phía (bipartite graph).

Hàm mục tiêu cần tối thiểu hóa được biểu diễn như sau:

$$\min \sum_{i=1}^M \sum_{j=1}^N C_{i,j} X_{i,j} \quad (2.16)$$

Thỏa mãn các điều kiện ràng buộc:

$$\begin{cases} \sum_{j=1}^N X_{i,j} \leq 1, & \forall i = 1 \dots M \\ \sum_{i=1}^M X_{i,j} \leq 1, & \forall j = 1 \dots N \\ X_{i,j} \in \{0, 1\} \end{cases} \quad (2.17)$$

Trong đó:

- $X_{i,j}$ là biến quyết định nhị phân: $X_{i,j} = 1$ nếu quỹ đạo t_i được ghép với detection d_j , và $X_{i,j} = 0$ nếu ngược lại.
- Các điều kiện ràng buộc đảm bảo nguyên tắc "độc nhất": mỗi quỹ đạo chỉ được cập nhật bởi tối đa một detection, và mỗi detection chỉ được gán cho tối đa một quỹ đạo.
- $C_{i,j}$ là ma trận chi phí (cost matrix), biểu thị mức độ "không tương đồng" giữa t_i và d_j .

Trong hệ thống sử dụng ByteTrack, chúng tôi sử dụng độ đo không gian IoU (Intersection over Union) để xây dựng ma trận chi phí thay vì dùng khoảng cách Euclid hay Mahalanobis, nhằm tận dụng tối đa khả năng định vị chính xác của YOLOv8:

$$C_{i,j} = 1 - \text{IoU}(\text{Box}(t_i), \text{Box}(d_j)) \quad (2.18)$$

Giá trị $C_{i,j}$ càng nhỏ đồng nghĩa với việc vùng bao dự đoán của track và vùng bao phát hiện thực tế chồng lấn càng nhiều, khả năng chúng là cùng một đối tượng càng cao.

Để giải quyết bài toán tối ưu tổ hợp này, chúng tôi sử dụng **thuật toán Hungarian** (hay còn gọi là thuật toán Kuhn-Munkres). Thuật toán này đảm bảo tìm ra nghiệm tối ưu toàn cục với độ phức tạp thời gian là $O(n^3)$, hoàn toàn đáp ứng được yêu cầu xử lý thời gian thực của hệ thống.

Thuật toán Hungarian (Kuhn–Munkres)

Thuật toán Hungarian giải bài toán phân công tuyến tính bằng cách tìm một phép ghép 1-1 tối ưu giữa tập track và detection trên ma trận chi phí $C \in \mathbb{R}^{n \times n}$.

Nếu ma trận không vuông ($M \neq N$) thì bổ sung hàng/cột giả để đưa về kích thước $n = \max(M, N)$; nếu cần cực đại hóa, chuyển sang cực tiểu hóa bằng $C' = C_{\max} - C$ hoặc dùng $-C$.^[7]

Quy trình chuẩn của thuật toán gồm các bước sau:

1. **Giảm hàng:** trừ phần tử nhỏ nhất của mỗi hàng cho toàn bộ hàng.
2. **Giảm cột:** trừ phần tử nhỏ nhất của mỗi cột cho toàn bộ cột.
3. **Phủ các số 0:** dùng số đường thẳng tối thiểu (ngang/dọc) để phủ hết các số 0 trong ma trận.
4. Nếu số đường phủ bằng n , ta chọn tập số 0 sao cho mỗi hàng/cột chỉ chọn đúng một phần tử (tương ứng với phép gán tối ưu). Nếu chưa đủ, tìm phần tử nhỏ nhất chưa bị phủ là m , trừ m cho mọi phần tử chưa phủ và cộng m cho các phần tử ở giao điểm của hai đường phủ; sau đó lặp lại bước 3.

Kết quả là ánh xạ ghép cặp tối ưu toàn cục với độ phức tạp $O(n^3)$, phù hợp cho bài toán theo dõi thời gian thực.

2.4.3 Thuật toán ByteTrack

Trong các phương pháp theo dõi đa đối tượng truyền thống (như DeepSORT hay SORT), một ngưỡng tin cậy (confidence threshold) cao thường được áp dụng để lọc bỏ các nhiễu nền (background noise). Ví dụ, chỉ những bounding box có điểm tin cậy > 0.5 mới được giữ lại để thực hiện liên kết. Tuy nhiên, trong bối cảnh giao thông thực tế, các đối tượng xe máy hoặc ô tô khi bị che khuất một phần (partial occlusion) hoặc bị mờ do chuyển động nhanh (motion blur) thường có điểm tin cậy giảm xuống thấp (ví dụ: $0.1 - 0.4$). Việc loại bỏ thẳng tay các detection này dẫn đến hệ quả là các quỹ đạo (tracks) bị đứt gãy, gây ra hiện tượng chuyển đổi ID (ID Switch) sai lệch.

Để giải quyết vấn đề này, chúng tôi sử dụng ByteTrack (Zhang et al., 2022). Đây là một thuật toán tracking đơn giản nhưng hiệu quả vượt trội, dựa trên tư tưởng: *"Tận dụng mọi detection, kể cả những detection có điểm tin cậy thấp, thay vì loại bỏ chúng"*.

Quy trình liên kết dữ liệu (data association) của ByteTrack được thực hiện qua cơ chế phân tầng (hierarchical matching) gồm các bước chi tiết sau:

Phân loại Detection

Tại mỗi khung hình t , tập hợp các detection thu được từ YOLOv8 được chia làm hai tập con dựa trên ngưỡng phân loại τ (thường là 0.5):

- $\mathcal{D}_{high}$: Tập các detection có độ tin cậy cao ($score > \tau$).
- $\mathcal{D}_{low}$: Tập các detection có độ tin cậy thấp ($\epsilon < score \leq \tau$), trong đó ϵ là ngưỡng tối thiểu (ví dụ 0.1) để loại bỏ hoàn toàn nhiễu.

Quy trình liên kết hai giai đoạn

Giai đoạn 1 (High Score Matching):

Hệ thống ưu tiên liên kết các detection chất lượng cao ($\mathcal{D}_{high}$) với các quỹ đạo hiện có (existing tracks) thông qua bộ lọc Kalman.

- Sử dụng bộ lọc Kalman để dự đoán vị trí của các Track ở khung hình hiện tại.
- Tính toán ma trận độ tương đồng giữa $\mathcal{D}_{high}$ và các Track dựa trên chỉ số IoU

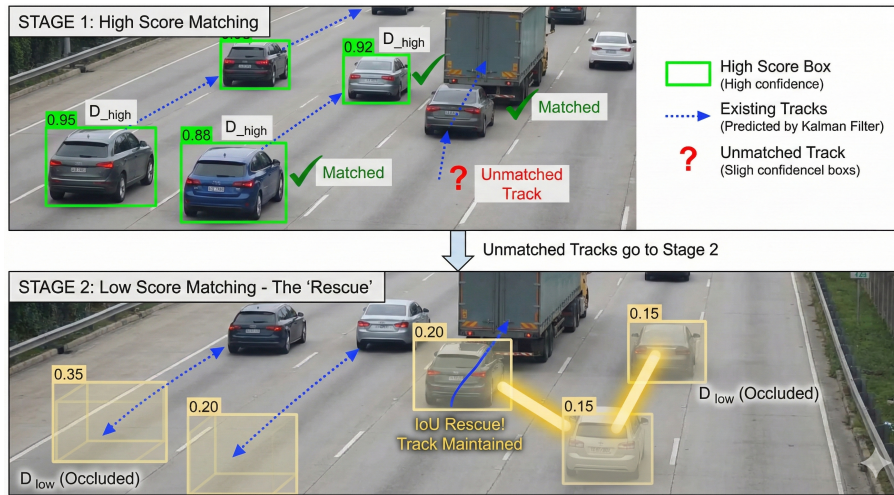
(Intersection over Union).

- Sử dụng thuật toán Hungarian để ghép cặp tối ưu.
- *Kết quả*: Các detection được ghép thành công sẽ cập nhật trạng thái cho Track tương ứng. Các detection chưa được ghép sẽ được giữ lại để khởi tạo Track mới. Các Track chưa tìm thấy đối tượng ghép (Unmatched Tracks) sẽ được chuyển sang giai đoạn 2.

Giai đoạn 2 (Low Score Matching):

Đây là bước đột phá của ByteTrack. Hệ thống cố gắng "cứu" các Track bị bỏ sót ở giai đoạn 1 bằng cách tìm kiếm trong tập detection độ tin cậy thấp ($\mathcal{D}_{low}$).

- Giả định rằng các Track chưa được ghép nối ở Giai đoạn 1 thường là do đối tượng bị che khuất, làm giảm confidence score của mô hình detection.
- Thực hiện ghép nối giữa các Unmatched Tracks (từ Giai đoạn 1) với $\mathcal{D}_{low}$, cũng sử dụng IoU làm trọng số.
- *Kết quả*: Nếu ghép nối thành công, Track đó sẽ được duy trì (giữ nguyên ID cũ) dù điểm detection thấp. Điều này giúp quỹ đạo liên mạch ngay cả khi xe đi khuất sau xe khác.



Hình 2.3: Minh họa quy trình liên kết hai giai đoạn của ByteTrack

Quản lý vòng đời Track

- **Track Birth (Sinh)**: Các detection trong $\mathcal{D}_{high}$ không ghép được với bất kỳ track nào ở Giai đoạn 1 sẽ được xem là đối tượng mới xuất hiện và được cấp ID mới.
- **Track Death (Hủy)**: Các Track không ghép được với bất kỳ detection nào (cả high và low) trong K khung hình liên tiếp (ví dụ $K = 30$) sẽ bị coi là đã ra khỏi khung hình và bị xóa bỏ.

Ưu điểm đối với bài toán giao thông Việt Nam

Việc áp dụng ByteTrack mang lại hai lợi ích lớn cho đề tài:

1. **Xử lý che khuất**: Trong điều kiện giao thông hỗn hợp tại Việt Nam, xe máy thường xuyên bị xe buýt hoặc ô tô che khuất. ByteTrack giúp "gấp" lại các xe

máy này ở giai đoạn 2, giảm thiểu việc đếm sai lưu lượng.

2. **Giảm tải tính toán:** ByteTrack chỉ sử dụng thông tin hình học (geometric information) để liên kết mà không cần trích xuất đặc trưng ngoại hình (appearance features) bằng mạng nơ-ron sâu như DeepSORT. Điều này giúp giảm tải tính toán cho GPU và phù hợp triển khai thời gian thực.

2.5 Xử lý ảnh và Hình học tính toán

Để giải quyết các bài toán cụ thể như nhận diện trạng thái đèn giao thông và xác định hành vi phương tiện (lấn làn, vượt đèn), hệ thống kết hợp các kỹ thuật xử lý ảnh trên không gian màu và các thuật toán hình học tính toán.

2.5.1 Không gian màu HSV trong nhận diện tín hiệu đèn

Chúng tôi chuyển đổi ảnh từ không gian màu RGB sang HSV để tách biệt thông tin màu sắc (Hue) khỏi độ sáng (Value), giúp thuật toán bền vững với thay đổi ánh sáng môi trường.

Quá trình chuyển đổi từ một điểm ảnh (R, G, B) sang (H, S, V) được thực hiện như sau. Đầu tiên, chuẩn hóa các giá trị $R, G, B \in [0, 1]$. Đặt $C_{max} = \max(R, G, B)$, $C_{min} = \min(R, G, B)$ và $\Delta = C_{max} - C_{min}$.

- **Value (V):** Độ sáng của điểm ảnh.

$$V = C_{max} \quad (2.19)$$

- **Saturation (S):** Độ bão hòa màu.

$$S = \begin{cases} 0 & \text{nếu } C_{max} = 0 \\ \frac{\Delta}{C_{max}} & \text{nếu } C_{max} \neq 0 \end{cases} \quad (2.20)$$

- **Hue (H):** Góc sắc độ (thể hiện màu sắc thuần túy).

$$H = \begin{cases} 60^\circ \times \left(\frac{G-B}{\Delta} \bmod 6 \right) & \text{nếu } C_{max} = R \\ 60^\circ \times \left(\frac{B-R}{\Delta} + 2 \right) & \text{nếu } C_{max} = G \\ 60^\circ \times \left(\frac{R-G}{\Delta} + 4 \right) & \text{nếu } C_{max} = B \end{cases} \quad (2.21)$$

Hệ thống sử dụng các ngưỡng (threshold) trên kênh H và S để đếm số lượng pixel thuộc về màu Đỏ, Vàng, hoặc Xanh. Trạng thái đèn được xác định bởi màu có số lượng pixel vượt trội (dominant color).

2.5.2 Hình học tính toán trong phát hiện vi phạm

Kiểm tra xe trong làn đường

Làn đường là đa giác (không bắt buộc lồi) $P = \{v_1, \dots, v_n\}$. Tâm xe $C(x, y)$ được kiểm tra bằng hàm `cv2.pointPolygonTest` (tương đương ray casting): giá trị trả về ≥ 0 nếu điểm nằm trong/biên, < 0 nếu ngoài.

Khoảng cách tới vạch dừng và sự kiện cắt qua

Vạch dừng: đoạn $P_1(x_1, y_1) - P_2(x_2, y_2)$. Khoảng cách vuông góc (đường thẳng qua P_1, P_2):

$$d(C, P_1P_2) = \frac{|(x_2 - x_1)(y_1 - y_C) - (x_1 - x_C)(y_2 - y_1)|}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}$$

Trong code, khoảng cách được "kẹp" vào đoạn (nếu hình chiếu ra ngoài đoạn, lấy khoảng cách tới đầu mút). Sự kiện vượt vạch khi: (i) $d < \delta$ (mặc định 15–20 px), (ii) track ID chưa nằm trong tập xe đã qua vạch.

Ước lượng hướng di chuyển dựa trên góc tham chiếu

Hướng di chuyển đi thẳng được xác định bởi góc tham chiếu θ_{ref} (đơn vị: độ). Giá trị này mặc định là 90° và được cập nhật khi hệ thống thiết lập Reference Vector.

Quá trình ước lượng dựa trên lịch sử vị trí bao gồm tối đa 10 điểm dữ liệu gần nhất. Để đảm bảo độ tin cậy, hệ thống yêu cầu tối thiểu 5 điểm dữ liệu và khoảng cách dịch chuyển phải lớn hơn 30 pixel. Từ vector chuyển động $\vec{v} = (dx, dy)$, góc di chuyển tuyệt đối θ và góc lệch tương đối θ_{rel} được tính toán như sau:

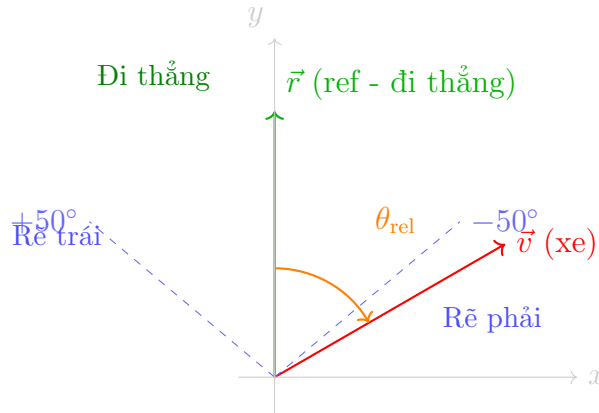
$$\theta = \text{atan2}(dy, dx) \quad (\text{độ})$$

$$\theta_{\text{rel}} = \theta - \theta_{\text{ref}}, \quad \text{với } \theta_{\text{rel}} \in [-180^\circ, 180^\circ]$$

Phân loại hướng di chuyển với ngưỡng 50° :

$$\begin{cases} \text{Đi thẳng} & |\theta_{\text{rel}}| \leq 50^\circ \\ \text{Rẽ phải} & \theta_{\text{rel}} < -50^\circ \\ \text{Rẽ trái} & \theta_{\text{rel}} > 50^\circ \end{cases}$$

Giải thích lựa chọn ngưỡng 50° : Ngưỡng 50° được chọn để phù hợp với đặc thù giao thông hỗn hợp nơi xe thường di chuyển theo nhiều hướng khác nhau tại các ngã tư. Ngưỡng này đủ rộng để hạn chế nhận diện sai các xe đi thẳng nhưng có độ lệch nhỏ do điều kiện đường sá, đồng thời đủ chặt để phân biệt rõ ràng giữa đi thẳng và rẽ.



Hình 2.4: So sánh góc chuyển động $\vec{v}$ với góc tham chiếu $\vec{r}$ (ngưỡng 50°). Vùng màu xanh lá: đi thẳng, vùng ngoài: rẽ trái/phải.

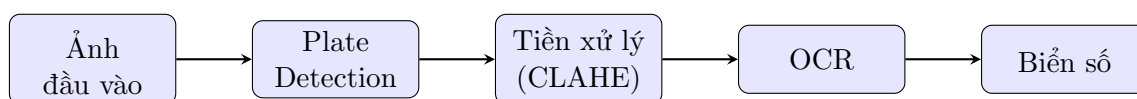
Chương 3

Cơ sở lý thuyết: Nhận diện biển số xe

3.1 Tổng quan về ALPR

Hệ thống nhận diện biển số xe tự động (Automatic License Plate Recognition - ALPR) thường bao gồm hai giai đoạn chính:

1. **Plate Detection:** Xác định vị trí biển số trong ảnh
2. **Character Recognition (OCR):** Nhận dạng các ký tự trên biển số



Hình 3.1: Pipeline tổng quan của hệ thống ALPR

3.2 Biển số xe Việt Nam

3.2.1 Các định dạng biển số

Biển số xe Việt Nam có các định dạng chính sau:

Bảng 3.1: Các định dạng biển số xe Việt Nam

Loại	Định dạng	Ví dụ
Xe ô tô (1 dòng)	XX-Y XXXXX	30G-07784
Xe ô tô (2 dòng)	XX-Y / XXXXX	30G / 07784
Xe máy	XX-YY / XXX.XX	29-H1 / 234.56

Trong đó:

- 2 chữ số đầu: Mã tỉnh/thành phố (VD: 30 = Hà Nội, 29 = Hà Nội cũ, 51 = TP.HCM)
- Chữ cái: Ký hiệu seri (A-Z, trừ I, O, Q)
- 5 chữ số cuối: Số thứ tự đăng ký

3.2.2 Thách thức nhận diện biển số Việt Nam

Việc nhận diện biển số Việt Nam gặp một số thách thức đặc thù:

- **Đa dạng định dạng:** Biển số có thể là 1 dòng hoặc 2 dòng, với các kích thước khác nhau
- **Font chữ đặc thù:** Font chữ trên biển số Việt Nam có một số điểm khác biệt
- **Điều kiện thực tế:** Biển số thường bị bẩn, mờ, hoặc nghiêng do góc quay camera
- **Ánh sáng:** Độ phản chiếu của biển số thay đổi theo điều kiện ánh sáng

3.3 PaddleOCR - Nhận dạng ký tự

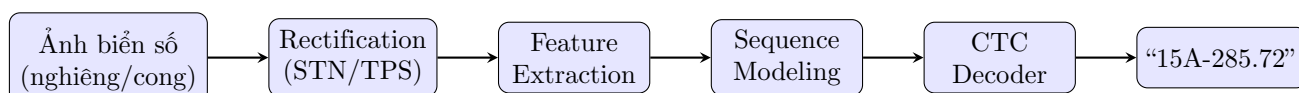
3.3.1 Giới thiệu PaddleOCR

PaddleOCR là một bộ công cụ OCR mã nguồn mở được phát triển bởi PaddlePaddle (Baidu). Trong dự án này, chúng tôi sử dụng **PaddleOCR phiên bản 2.7.x** (tương ứng với PP-OCRv4). PaddleOCR hỗ trợ nhận dạng văn bản đa ngôn ngữ (hơn 80 ngôn ngữ) với kiến trúc linh hoạt, phù hợp cho nhiều bài toán OCR.

Trong đề tài này, chúng tôi sử dụng PaddleOCR với tham số `use_textline_orientation=True` để tự động phát hiện và xoay văn bản nghiêng, kết hợp với `lang='en'` cho bộ ký tự Latin (phù hợp với biển số Việt Nam).

3.3.2 Pipeline nhận dạng văn bản trong PaddleOCR

PaddleOCR sử dụng một pipeline hoàn chỉnh để nhận dạng văn bản, bao gồm các bước chính:



Hình 3.2: Pipeline nhận dạng văn bản trong PaddleOCR

3.3.3 Rectification - Nắn thẳng biển số

Biển số xe trong thực tế thường bị nghiêng, xoay hoặc biến dạng do góc chụp camera. Để giảm ảnh hưởng của góc chụp và biến dạng, PaddleOCR sử dụng module **Rectification** để tự động nắn thẳng ảnh văn bản trước khi đưa vào mô hình nhận dạng.

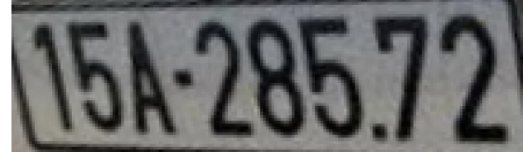
Spatial Transformer Network (STN)

STN (Spatial Transformer Network)^{[?]1} là một module học sâu cho phép mạng neural tự động học cách biến đổi không gian của ảnh đầu vào. STN bao gồm 3 thành phần:

1. **Localization Network:** Mạng CNN nhỏ học các tham số biến đổi θ từ ảnh đầu vào
2. **Grid Generator:** Tạo lưới tọa độ cho ảnh output dựa trên θ
3. **Sampler:** Lấy mẫu pixel từ ảnh gốc theo lưới tọa độ đã tính

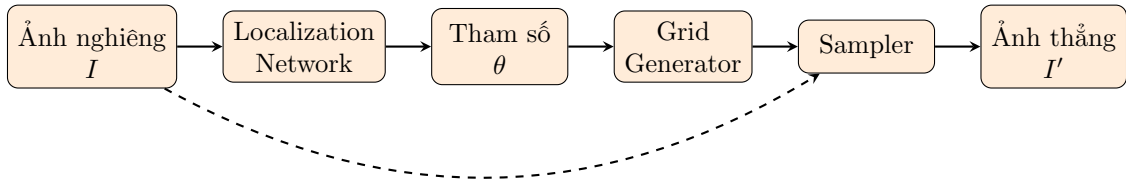


(a) Biển số bị nghiêng do góc camera



(b) Biển số sau khi nắn thẳng

Hình 3.3: Minh họa quá trình Rectification biển số xe



Hình 3.4: Kiến trúc Spatial Transformer Network

Phép biến đổi Affine: Với phép biến đổi affine 2D, ma trận tham số θ có dạng:

$$\theta = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \end{bmatrix} \quad (3.1)$$

Tọa độ điểm (x_i^s, y_i^s) trong ảnh nguồn được tính từ tọa độ (x_i^t, y_i^t) trong ảnh đích:

$$\begin{pmatrix} x_i^s \\ y_i^s \end{pmatrix} = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \end{bmatrix} \begin{pmatrix} x_i^t \\ y_i^t \\ 1 \end{pmatrix} \quad (3.2)$$

Phép biến đổi affine có thể biểu diễn: xoay (rotation), co giãn (scale), dịch chuyển (translation), và nghiêng (shear).

Thin-Plate Spline (TPS) Transformation

Đối với văn bản bị uốn cong (curved text), phép biến đổi affine không đủ linh hoạt. PaddleOCR sử dụng **TPS (Thin-Plate Spline)**^[7] để xử lý các biến dạng phi tuyến tính phức tạp hơn.

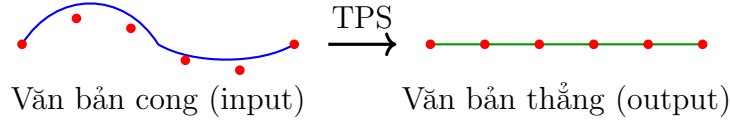
TPS sử dụng một tập các điểm điều khiển (control points) $\{(x_k, y_k)\}_{k=1}^K$ để định nghĩa phép biến đổi. Công thức TPS:

$$f(x, y) = a_0 + a_x x + a_y y + \sum_{k=1}^K w_k \cdot U(\|(x, y) - (x_k, y_k)\|) \quad (3.3)$$

Trong đó:

- a_0, a_x, a_y : Các tham số affine
- w_k : Trọng số của điểm điều khiển thứ k

- $U(r) = r^2 \log(r)$: Hàm cơ sở radial basis function (RBF)
- K : Số điểm điều khiển (thường $K = 20$ trong PaddleOCR)



Hình 3.5: TPS transformation nắn thẳng văn bản cong

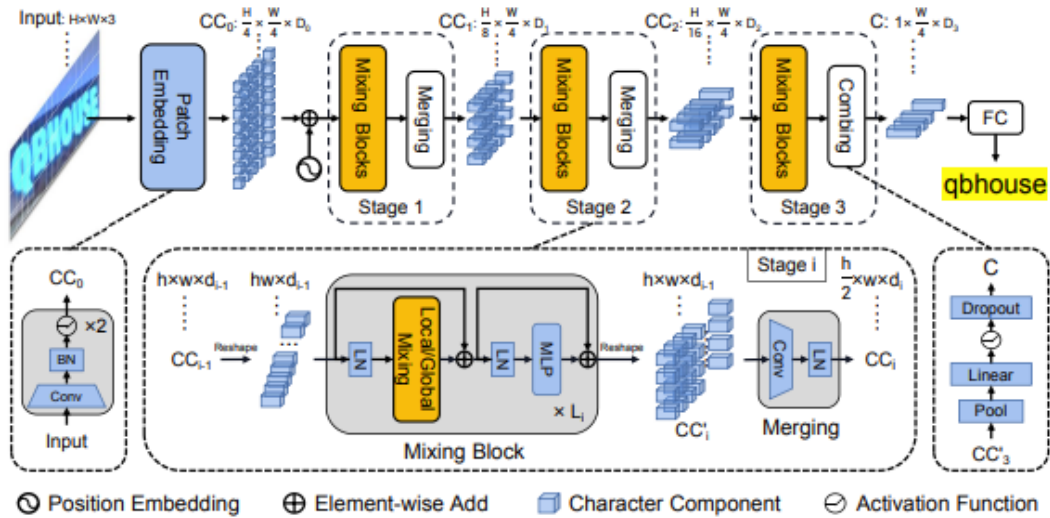
Ứng dụng trong PaddleOCR

Khi khởi tạo PaddleOCR với `use_textline_orientation=True`:

- Mô hình tự động phát hiện hướng của văn bản
- Áp dụng phép xoay để đưa văn bản về hướng ngang
- Với văn bản nghiêng/cong, module TPS được kích hoạt để nắn thẳng
- Ảnh đã được rectified được đưa vào mô hình nhận dạng chính

3.3.4 Kiến trúc SVTR - Scene Text Recognition

Từ phiên bản PP-OCrv3, PaddleOCR đã chuyển từ kiến trúc CRNN truyền thống sang **SVTR** (Single Visual model for Scene Text Recognition)^[?]. SVTR là một kiến trúc hiệu quả hơn, sử dụng hoàn toàn cơ chế attention thay vì RNN.



Hình 3.6: Kiến trúc tổng quan của SVTR (nguồn: Du et al., 2022)

Ý tưởng chính của SVTR

SVTR xử lý ảnh văn bản bằng cách:

1. **Patch Embedding**: Chia ảnh thành các patch nhỏ (character components)
2. **Hierarchical Mixing**: Trộn thông tin qua nhiều tầng với local và global attention

3. Linear Prediction: Dự đoán ký tự trực tiếp từ feature cuối



Hình 3.7: Kiến trúc SVTR trong PaddleOCR

Cơ chế cắt Patch và trích xuất đặc trưng

SVTR xử lý ảnh văn bản giống như Vision Transformer (ViT), nhưng được tối ưu cho văn bản:

Bước 1 - Patch Embedding:

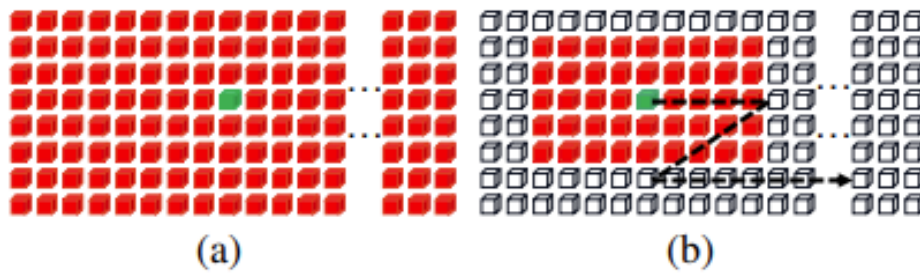
- Ảnh đầu vào kích thước $H \times W \times 3$ được chia thành các patch kích thước 4×4
- Mỗi patch được flatten và chiếu lên không gian D chiều
- Kết quả: chuỗi $T = \frac{H}{4} \times \frac{W}{4}$ token, mỗi token là vector D chiều

Bước 2 - Local Mixing (Intra-character):

- Học các đặc trưng **bên trong** từng ký tự (nét, đường cong, góc)
- Sử dụng window-based self-attention với window size nhỏ
- Cho phép các patch lân cận trao đổi thông tin

Bước 3 - Global Mixing (Inter-character):

- Học mối quan hệ **giữa các ký tự** (ngữ cảnh)
- Sử dụng global self-attention trên toàn bộ chuỗi
- Giúp phân biệt các ký tự giống nhau dựa trên ngữ cảnh



Hình 3.8: So sánh Local Mixing và Global Mixing trong SVTR

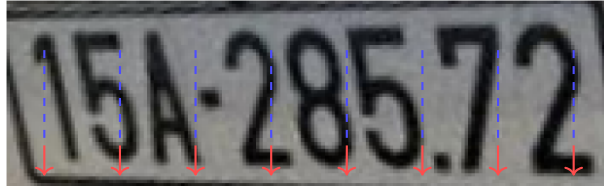
Bước 4 - Height Merging:

- Gộp các token theo chiều cao để tạo chuỗi 1D
- Giảm từ $\frac{H}{4} \times \frac{W}{4}$ thành $1 \times \frac{W}{4}$ token
- Mỗi token đại diện cho một “lát cắt” dọc của ảnh

So sánh SVTR và CRNN

3.3.5 Kiến trúc CRNN (Legacy)

Mặc dù SVTR là mặc định trong các phiên bản mới, PaddleOCR vẫn hỗ trợ kiến trúc CRNN truyền thống. CRNN (Convolutional Recurrent Neural Network)



Ảnh biển số được chia thành các “lát cắt” dọc
↓ mỗi lát → vector đặc trưng D chiều

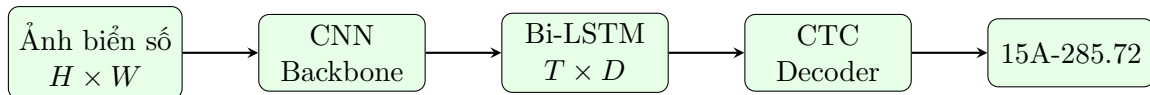
Hình 3.9: Minh họa cắt ảnh thành các lát dọc trong SVTR

Bảng 3.2: So sánh kiến trúc SVTR và CRNN

Tiêu chí	CRNN	SVTR
Trích xuất đặc trưng	CNN (ResNet, MobileNet)	Patch Embedding + Transformer
Mô hình hóa chuỗi	Bidirectional LSTM	Self-Attention (Local + Global)
Tính toán	Tuần tự	Song song
Ngữ cảnh	Hạn chế bởi hidden size LSTM	Toàn cục qua attention

bao gồm:

1. **CNN Backbone:** Trích xuất đặc trưng không gian từ ảnh đầu vào. Backbone thường sử dụng là MobileNet hoặc ResNet biến thể.
2. **Sequence Modeling (LSTM):** Mô hình hóa quan hệ tuần tự giữa các ký tự. Sử dụng Bidirectional LSTM để capture thông tin từ cả hai hướng.
3. **CTC Decoder:** Giải mã chuỗi ký tự cuối cùng từ output của LSTM.



Hình 3.10: Kiến trúc CRNN cho OCR trong PaddleOCR

3.3.6 CTC Decoder - Giải mã chuỗi ký tự

CTC (Connectionist Temporal Classification) là phương pháp giải mã cho phép huấn luyện mà không cần alignment giữa input và output. Đây là đặc điểm quan trọng vì ta không biết chính xác vị trí của từng ký tự trên ảnh.

Nguyên lý hoạt động

Sau khi đi qua backbone (SVTR hoặc CRNN), mỗi “lát cắt” dọc của ảnh được biểu diễn bằng một vector xác suất trên tập ký tự (bao gồm cả ký tự đặc biệt “blank” ϵ).

Ví dụ: Với ảnh biển số “15A”, output của mô hình có thể là:

Lát	1	2	3	4	5	6	7
Output	1	1	ϵ	5	ϵ	A	A

Sau khi áp dụng CTC collapse (loại bỏ lặp và blank): “11-5-AA” $\rightarrow$ “15A”

Công thức xác suất CTC

Xác suất của một chuỗi output $\mathbf{l}$ được tính:

$$P(\mathbf{l}|\mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} P(\pi|\mathbf{x}) \quad (3.4)$$

Trong đó:

- $\mathbf{x}$: Chuỗi feature vectors (từ các lát cắt)
- $\mathbf{l}$: Chuỗi ký tự mục tiêu (VD: “15A-285.72”)
- π : Một alignment path (VD: “1-5-A—2-8-5-...”)
- $\mathcal{B}^{-1}(\mathbf{l})$: Tập tất cả các path có thể tạo ra $\mathbf{l}$ sau khi loại bỏ ký tự blank và ký tự lặp

Ví dụ: Với chuỗi mục tiêu “AB”, các path hợp lệ có thể là:

- “A-A-B-B” $\rightarrow$ “AB” (loại bỏ lặp)
- “A-B-” $\rightarrow$ “AB” (loại bỏ blank ‘-’)
- “-A-B-” $\rightarrow$ “AB”

Greedy Decoding và Beam Search

Trong inference, CTC decoder sử dụng:

Greedy Decoding (mặc định):

- Tại mỗi timestep, chọn ký tự có xác suất cao nhất
- Sau đó áp dụng collapse rule
- Đơn giản, không duy trì nhiều giả thuyết

Beam Search:

- Duy trì k hypotheses tốt nhất tại mỗi bước
- Cộng dồn log-probability của các ký tự
- Mở rộng không gian giả thuyết trong quá trình giải mã

CTC Loss

CTC Loss được tính bằng negative log-likelihood:

$$\mathcal{L}_{CTC} = -\log P(\mathbf{l}|\mathbf{x}) \quad (3.5)$$

Để tính toán hiệu quả, sử dụng thuật toán forward-backward tương tự HMM.

3.4 Tiền xử lý ảnh biến số

Chất lượng ảnh biến số ảnh hưởng trực tiếp đến khả năng nhận dạng của OCR. Chúng tôi áp dụng các kỹ thuật tiền xử lý sau:

3.4.1 CLAHE - Cân bằng histogram thích ứng

CLAHE (Contrast Limited Adaptive Histogram Equalization) là kỹ thuật cải thiện độ tương phản ảnh, đặc biệt hiệu quả cho ảnh có độ sáng không đều.

Nguyên lý hoạt động

1. **Chia tile:** Chia ảnh thành các tile nhỏ (VD: 8×8 pixels)
2. **Histogram Equalization cục bộ:** Thực hiện histogram equalization trên từng tile
3. **Clip Limit:** Giới hạn độ tương phản (clip limit) để tránh khuếch đại noise. Nếu histogram bin vượt quá clip limit, phần dư sẽ được phân bố lại đều cho các bin khác.
4. **Bilinear Interpolation:** Nội suy bilinear để ghép các tile lại một cách mượt mà, tránh hiện tượng blocking artifact.

Công thức toán học

Công thức histogram equalization cho một pixel:

$$s_k = (L - 1) \cdot \sum_{j=0}^k p_r(r_j) = (L - 1) \cdot \text{CDF}(r_k) \quad (3.6)$$

Trong đó:

- s_k : Mức xám mới của pixel
- L : Số mức xám (256 cho ảnh 8-bit)
- $p_r(r_j)$: Xác suất xuất hiện của mức xám r_j
- CDF: Cumulative Distribution Function của histogram

Clip Limit trong CLAHE:

$$\beta = \frac{M \times N}{L} \times \text{clipLimit} \quad (3.7)$$

Trong đó $M \times N$ là kích thước tile và clipLimit là tham số (thường từ 2.0 đến 4.0).

3.4.2 Sharpening - Làm sắc nét

Bộ lọc làm sắc nét giúp tăng cường các cạnh của ký tự, làm cho chúng rõ ràng hơn trước khi đưa vào OCR.

Kernel Sharpening

Chúng tôi sử dụng kernel convolution để làm sắc nét:

$$K_{\text{sharpen}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (3.8)$$

Nguyên lý

Kernel này hoạt động bằng cách:

- Nhân pixel trung tâm với hệ số lớn (5)
- Trừ đi trung bình của 4 pixel lân cận
- Kết quả: tăng cường sự khác biệt giữa pixel và các pixel xung quanh $\rightarrow$ cạnh sắc nét hơn

Công thức tính pixel mới:

$$I'(x, y) = 5 \cdot I(x, y) - [I(x - 1, y) + I(x + 1, y) + I(x, y - 1) + I(x, y + 1)] \quad (3.9)$$

3.4.3 Quy trình tiền xử lý hoàn chỉnh

1. **Crop ảnh biển số:** Từ đầu ra detection của YOLOv8
2. **Chuyển Grayscale:** Để áp dụng CLAHE
3. **Áp dụng CLAHE:** Với clipLimit=2.0, tileGridSize=(8,8)
4. **Chuyển về RGB:** Để đưa vào PaddleOCR
5. **Sharpening (tùy chọn):** Áp dụng nếu ảnh còn mờ

3.5 So sánh các phiên bản YOLOv8

Trong đề tài này, chúng tôi sử dụng **YOLOv8n** (nano) cho cả phát hiện phương tiện và phát hiện biển số vì phù hợp với ràng buộc triển khai và tài nguyên.

Bảng 3.3: So sánh các phiên bản YOLOv8

Model	Params (M)	FLOPs (G)
YOLOv8n	3.2	8.7
YOLOv8s	11.2	28.6
YOLOv8m	25.9	78.9
YOLOv8l	43.7	165.2
YOLOv8x	68.2	257.8

Lý do chọn YOLOv8n:

- Số lượng tham số nhỏ (3.2M) → phù hợp triển khai trên phần cứng phổ thông
- FLOPs thấp (8.7G) → giảm chi phí tính toán
- Phù hợp cho bài toán phát hiện biển số một lớp và tích hợp trong pipeline tổng thể

3.6 Cơ chế Validation & Retry

3.6.1 Vấn đề với OCR đơn frame

OCR có thể cho chuỗi đầu ra khác nhau giữa các frame do:

- Biển số bị rung, blur khi xe di chuyển
- Ánh sáng thay đổi (phản chiếu, bóng)
- Góc nhìn thay đổi khi xe tiến lại gần camera
- Nhiều từ mô hình OCR

3.6.2 Giải pháp: Format Validation

Thay vì lưu trữ nhiều chuỗi đầu ra và voting, chúng tôi sử dụng phương pháp validate format biển số Việt Nam:

3.6.3 Retry Mechanism

Khi chuỗi OCR không đúng format, hệ thống sẽ retry ở frame tiếp theo:

- Chỉ lưu chuỗi **đã validate** (đúng format)

Algorithm 1: Thuật toán Validation biển số Việt Nam

Input: *ocr_text*: Chuỗi OCR, *vehicle_class*: Loại xe
Output: (*is_valid*, *cleaned_plate*): Trạng thái hợp lệ và chuỗi đã làm sạch

```
1 cleaned  $\leftarrow$  RemoveSpaces(ocr_text);
2 if vehicle_class  $\in$  {motorbike, bicycle} then
3   | pattern  $\leftarrow$  “[0-9]{2}[A-Z][A-Z0-9][0-9]{4,5}$”;
4   | min_len  $\leftarrow$  8, max_len  $\leftarrow$  9;
5 else
6   | pattern  $\leftarrow$  “[0-9]{2}[A-Z][0-9]{4,5}$”;
7   | min_len  $\leftarrow$  7, max_len  $\leftarrow$  8;
8 end
9 if min_len  $\leq$  len(cleaned)  $\leq$  max_len AND Match(pattern, cleaned)
   | then
10  | return (True, cleaned);
11 end
12 return (False, cleaned);
```

- Nếu OCR sai format hoặc không đọc được $\rightarrow$ không lưu, thử lại frame sau
- Khi đã có chuỗi hợp lệ $\rightarrow$ không xử lý OCR nữa (tiết kiệm tài nguyên)

$$\text{ShouldRunOCR}(\text{track_id}) = \begin{cases} \text{True} & \text{nếu chưa có chuỗi valid} \\ \text{False} & \text{nếu đã có chuỗi valid} \end{cases} \quad (3.10)$$

3.7 Character Correction cho biển số Việt Nam

3.7.1 Các lỗi phổ biến của OCR

Do một số ký tự có hình dáng tương tự, PaddleOCR có thể nhầm lẫn:

Bảng 3.4: Bảng hiệu chỉnh ký tự phổ biến

Sai	Đúng	Điều kiện
0 (số không)	O (chữ O)	Vị trí chữ cái (index 2)
O (chữ O)	0 (số không)	Vị trí chữ số
1 (số một)	I (chữ I)	Vị trí chữ cái
8 (số tám)	B (chữ B)	Vị trí chữ cái
5 (số năm)	S (chữ S)	Vị trí chữ cái

3.7.2 Logic hiệu chỉnh dựa trên định dạng

Dựa trên cấu trúc biển số Việt Nam (ví dụ: 30G-07784):

- **Vị trí 0-1:** Bắt buộc là chữ số (mã tỉnh)
- **Vị trí 2:** Bắt buộc là chữ cái (seri)
- **Vị trí 3-7:** Bắt buộc là chữ số (số thứ tự)

Thuật toán hiệu chỉnh:

1. Kiểm tra độ dài chuỗi OCR
2. Tại mỗi vị trí, kiểm tra xem ký tự có đúng loại (chữ/số) không
3. Nếu sai loại, tra bảng mapping để sửa

3.8 Hai phương pháp triển khai ALPR

Trong hệ thống của chúng tôi, có hai cách tiếp cận để thực hiện nhận diện biển số xe:

3.8.1 Phương pháp 1: Two-Stage ALPR (Hai giai đoạn)

Đây là phương pháp truyền thống, chia quá trình thành hai bước độc lập:

Kiến trúc

1. **Giai đoạn 1 - Phát hiện phương tiện:** Sử dụng YOLOv8n (5 lớp) để phát hiện các loại xe (xe máy, xe đạp, ô tô, xe buýt, xe tải)
2. **Giai đoạn 2 - Phát hiện biển số:** Crop vùng xe từ giai đoạn 1, chạy qua YOLOv8n chuyên biệt (1 lớp: license_plate) để phát hiện biển số
3. **Giai đoạn 3 - OCR:** Crop vùng biển số, áp dụng tiền xử lý (CLAHE, sharpening), sau đó sử dụng PaddleOCR để nhận dạng ký tự

Ưu điểm

- **Mô hình chuyên biệt:** Tách riêng nhiệm vụ phát hiện biển số để tập trung đặc trưng mục tiêu
- **Tập trung đặc trưng:** Chỉ xử lý một nhiệm vụ (plate detection) nên dễ tối ưu dữ liệu và huấn luyện
- **Linh hoạt:** Có thể thay thế từng module độc lập (ví dụ: thay YOLOv8 bằng Faster R-CNN cho plate detection)

Nhược điểm

- **Nhiều lần suy luận:** Cần 2 lần inference qua YOLO (vehicle detection + plate detection)
- **Nhiều mô hình:** Phải load 2 mô hình vào GPU/RAM
- **Phức tạp:** Logic phải quản lý 2 models, crop vùng xe, rồi mới detect plate

3.8.2 Phương pháp 2: One-Stage Semi-supervised (Một giai đoạn)

Phương pháp này tích hợp việc phát hiện phương tiện và biển số vào một mô hình duy nhất.

Kiến trúc

1. **Giai đoạn 1 - Phát hiện đa lớp:** Sử dụng YOLOv8n (6 lớp) để phát hiện đồng thời cả xe và biển số trong một lần forward pass
2. **Giai đoạn 2 - Matching:** Sử dụng thuật toán containment để gán biển số cho xe (kiểm tra xem bbox biển có nằm trong bbox xe không)

3. **Giai đoạn 3 - OCR:** Tương tự Phương pháp 1, crop vùng biển số và nhận dạng ký tự

Quy trình gán nhãn bán giám sát (Semi-supervised)

Để có được mô hình 6 lớp, chúng tôi áp dụng kỹ thuật Semi-supervised Labeling:

1. **Chuẩn bị Teacher Model:** Sử dụng mô hình phát hiện biển số từ Phương pháp 1 làm teacher
2. **Tự động gán nhãn:** Chạy teacher model trên toàn bộ dataset vietnam_vehicle_dataset (chỉ có 5 lớp xe) để tự động phát hiện và gán nhãn biển số
3. **Hợp nhất dataset:** Merge các annotation biển số vào file gốc, tạo thành dataset 6 lớp (5 loại xe + license_plate)
4. **Huấn luyện lại:** Train YOLOv8n từ đầu trên dataset 6 lớp này

Ưu điểm

- **Một lần suy luận:** Chỉ cần 1 lần inference qua YOLO
- **Một mô hình:** Chỉ cần load 1 mô hình
- **Mapping tự động:** Dễ dàng gán biển số cho xe thông qua kiểm tra containment
- **Học ngữ cảnh:** Mô hình học được mối quan hệ không gian giữa xe và biển số

Nhược điểm

- **Mất cân bằng lớp:** Biển số là lớp hiếm, cần xử lý cân bằng dữ liệu
- **Tỷ lệ đối tượng nhỏ:** Biển số chiếm tỷ lệ nhỏ trong dataset, cần chú ý khi huấn luyện
- **Phụ thuộc teacher model:** Chất lượng annotation tự động phụ thuộc vào chất lượng teacher model

3.8.3 So sánh hai phương pháp

Trong phạm vi lý thuyết, so sánh dựa trên cấu trúc pipeline và chi phí tính toán dự kiến.

- **Số lần suy luận:** One-Stage một lần forward pass; Two-Stage hai lần phát hiện (xe + biển số) rồi OCR.
- **Số mô hình:** Two-Stage dùng hai mô hình phát hiện; One-Stage dùng một mô hình đa lớp.
- **Mức độ chuyên biệt:** Two-Stage tách riêng nhiệm vụ; One-Stage gộp nhiệm vụ trong một mô hình.
- **Độ phức tạp triển khai:** One-Stage đơn giản hơn trong vận hành và bảo trì; Two-Stage phức tạp hơn do nhiều module.

Ưu tiên triển khai: Tùy ràng buộc hạ tầng và mục tiêu hệ thống, lựa chọn One-Stage hoặc Two-Stage cho phù hợp.